Cache memories

M. Sonza Reorda

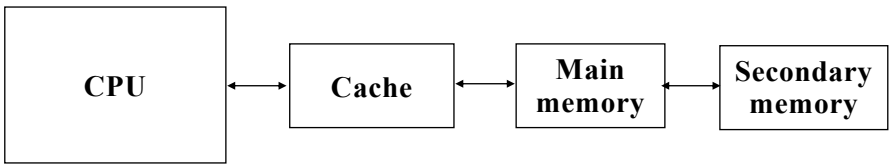
Politecnico di Torino
Dip. di Automatica e Informatica



1

Introduction

Cache memories are small but high-speed memories that are interposed between the processor and the main memory.



2

Locality of reference

The presence of a cache can improve the performance of a system due to the locality of references observed in most programs.

Locality takes two forms:

- ***Temporal locality*: if at time `t` the program accesses to a given memory cell, it is highly probable that the program accesses again the same cell by the time `t + Δ`**
- ***Spatial locality*: if at time `t` the program accesses a memory cell with address `x`, it is highly likely that by the time `t + Δ` the program will also access the memory cell with address `x ± e`.**

3

Working principle

If the entire block is loaded in the cache at time `t` (first access to a memory block), it is likely that for a certain time `Δ` the program will find in the cache all of the words it needs.

4

Performance

Let define the following entities:

- ***h*: cache hit ratio**
- ***C*: cache access time**
- ***M*: memory access time when the data is not in the cache.**

The average access time to memory is

$$t_{ave} = hC + (1-h)M$$

Normal values for *h* are in the order of 0.9.

5

Cache organization

A cache is organized in lines.

A line contains a memory block.

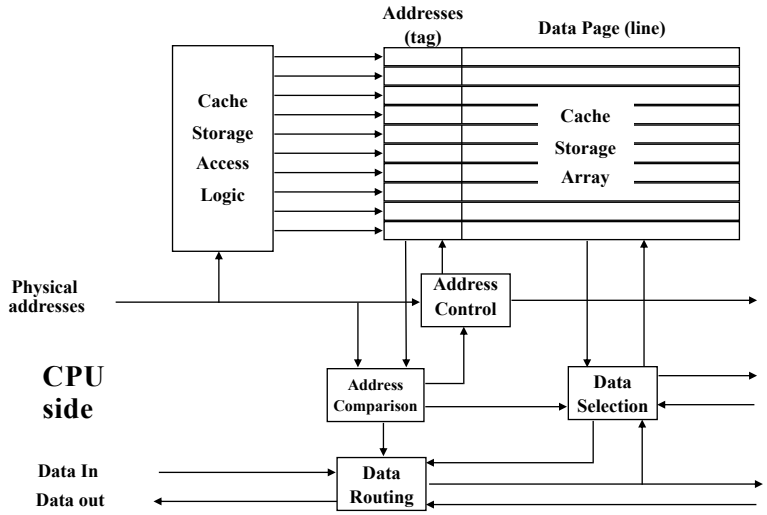
Each line is associated with a tag field, which indicates the memory block present in the line at that time.

The cache also contains the logic for

- **intercepting the addresses produced by the processor**
- **checking inside the cache the possible presence of the block that the processor wants to access**
- **possibly loading the block.**

6

Architecture



7

Cache behavior

The cache is located between the processor and the main memory.

Each time the processor performs an access to memory the cache intercepts the address and checks if the block to which the word belongs is in the cache, checking the value of the tags

- **if yes: it extracts the word from the block and provides it to the CPU without any access to the main memory (*hit*)**
- **if no: it loads in the cache the entire block that the word is part of (*miss*).**

8

Performance

In the case of a hit, the cache reduces the memory access times by a factor that is dependent on the ratio between cache and primary memory access times.

In case of a miss, the cache responds in two possible ways:

- **It accesses the memory and loads the entire missing block; then it provides the requested word. The access time is therefore higher than in a cache-free system.**
- **It accesses the memory and immediately provides the requested word (*load-through* or *early restart*). This technique requires a higher cost in terms of cache hardware, but miss situations have a more limited impact on cache performance.**

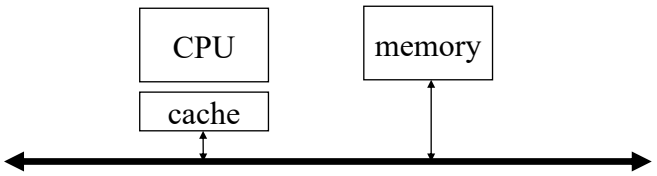
9

Cache position

The cache is normally located between the CPU and the bus, rather than between the main memory and the bus.

The benefits that are obtained in this way are:

- **the bus load is reduced**
- **the solution is compatible with a multiprocessor architecture.**



10

Instruction Cache and Data Cache

In some cases there are separate caches for data and instructions; in other cases there is only one for instructions and data.

The cache for instructions is typically easier to handle than data, as the instructions can not be changed (no write operations on the instruction cache).

11

Harvard architecture

If there are two caches, the architecture of the system falls into the scheme known as *Harvard architecture*, characterized by the existence of two separate data and code memories.

Harvard architecture contrasts with *von Neumann architecture*.

12

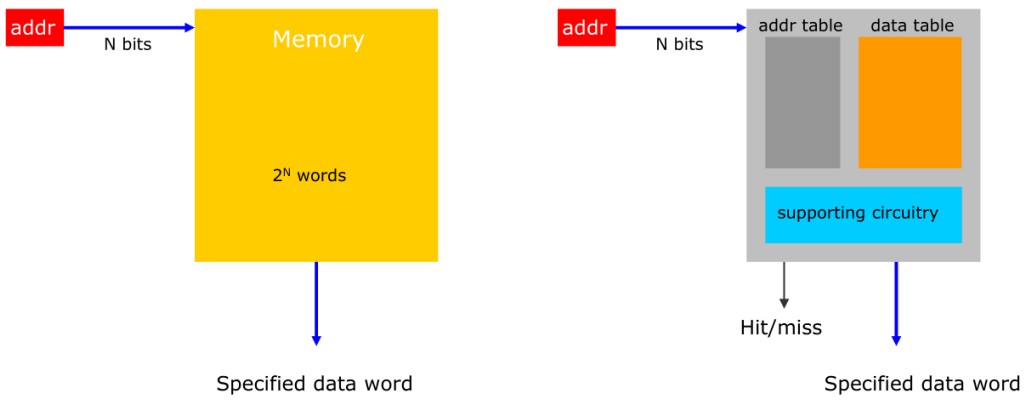
Characteristic parameters

They are:

- Cache size
- Mapping
- Replacing algorithm
- Main memory update mechanism
- Block size.

13

Organization



14

Cache size

Choosing the optimal size of the cache is very important for the system cost and the performance.

As the size increases

- **The cost increases**
- **The system performance improves**
- **The cache itself becomes slower.**

Frequent sizes range from a few KB to a few MBs.

15

Mapping

The mapping mechanism defines in which line of the cache a certain memory block is written when it is uploaded in the cache.

You must ensure (at an acceptable cost) that the cache can quickly verify if it contains the data corresponding to a certain address.

Possible solutions are:

- **associative memory (too expensive)**
- **sequential scan (too slow)**
- **address mapping mechanism (normally used).**

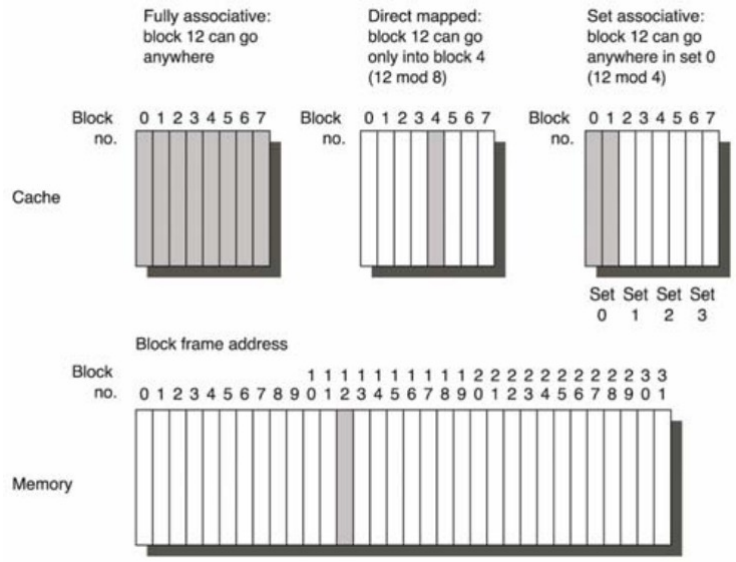
16

Mapping

The 3 main mapping mechanisms are:

- ***direct mapping***
- ***associative mapping*** **(rarely adopted)**
- ***set associative mapping.***

17



18

Direct Mapping

Each memory block *i* is statically associated to a line *k* in the cache using the expression

$$k = i \bmod N$$

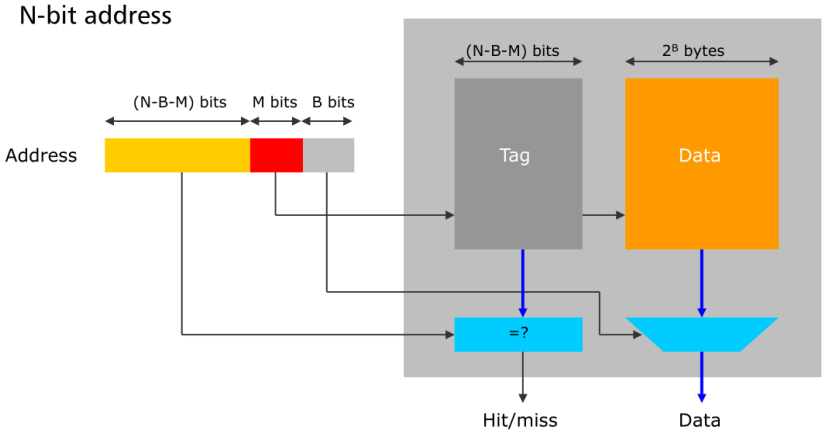
where *N* is the number of lines in the cache.

The computation of *k* can be easily performed by just taking the least significant bits in the value *i*.

19

Direct Mapping

- 2^B byte block
- 2^M entries
- N-bit address

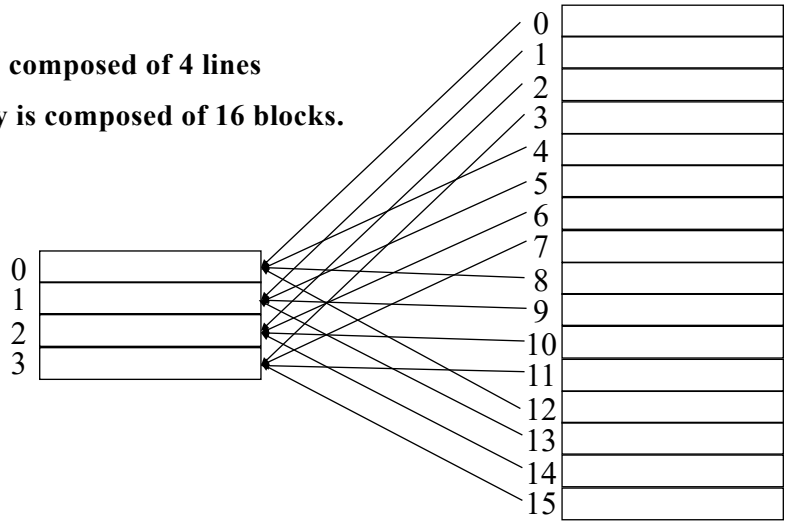


20

Direct Mapping: example

Let assume that

- **The cache is composed of 4 lines**
- **The memory is composed of 16 blocks.**



21

Direct Mapping

Advantages:

- **the mechanism can be easily implemented in hardware (the least significant block identifier bits identify the cache line)**

Disadvantages:

- **if the program frequently accesses 2 blocks corresponding to the same cache line, a miss occurs at each memory access.**

22

Associative Mapping

Each block of the main memory can be stored in any cache block.

Advantages:

- **maximum flexibility in choosing the cache block to use**

Disadvantages:

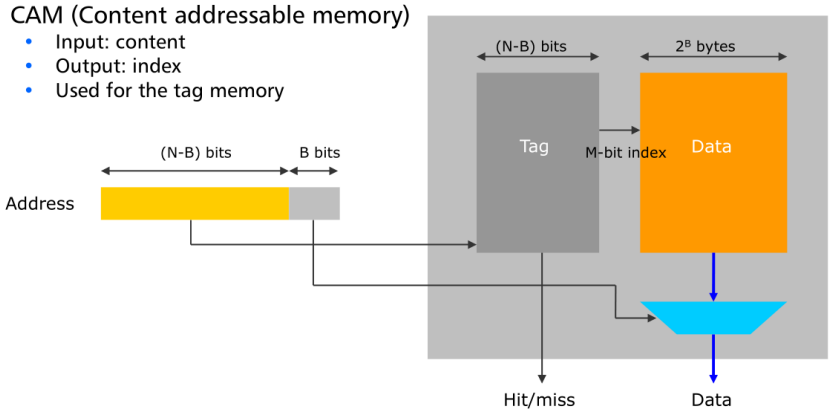
- **complexity of search hardware (usually adopting associative memory).**

23

(Fully) Associative Mapping

- 2^B byte block
- 2^M entries
- N-bit address

- CAM (Content addressable memory)
  - Input: content
  - Output: index
  - Used for the tag memory



24

Set Associative Mapping

Characteristics:

- **the cache lines are subdivided into *S* sets, each consisting of *W* lines**
- **a block *i* is associated with the set *k*, with *k* = *i mod s***
- **the block *i* can be put into any of the *W* lines of the k set.**

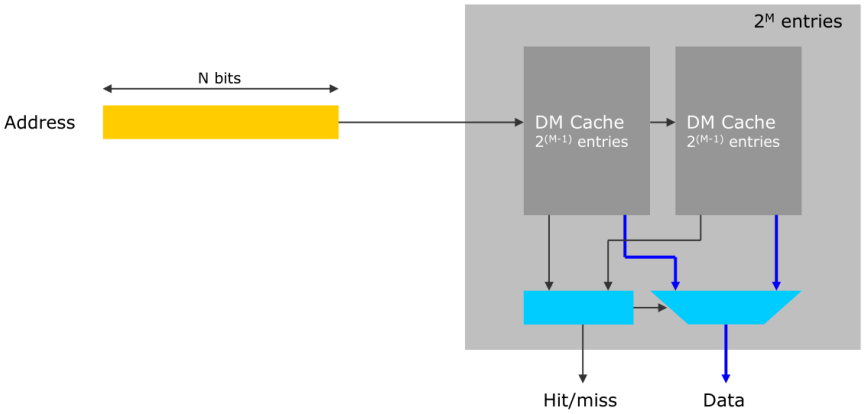
A set associative cache with *W* lines in each set is called a *W-ways* cache.

Common values for *W* are 2 and 4.

25

2-Way Set Associative Example

- 2^B byte block
- 2^M entries
- N-bit address



26

Cache memories

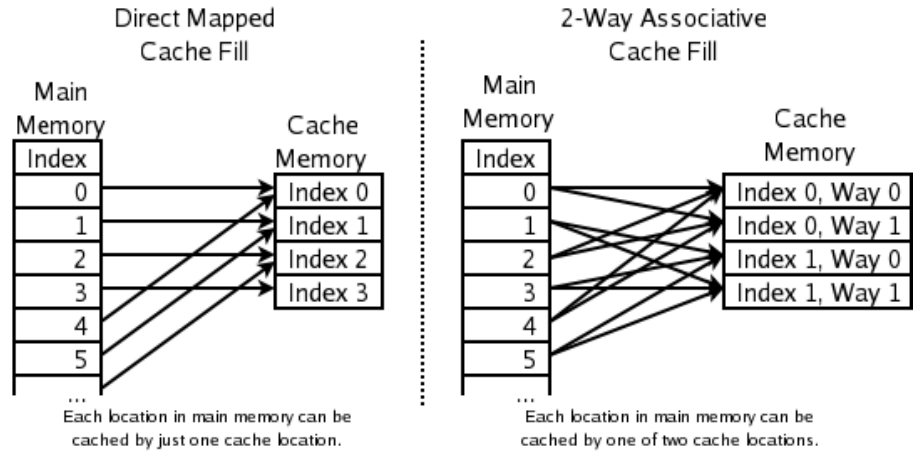
M. Sonza Reorda

Politecnico di Torino
Dip. di Automatica e Informatica



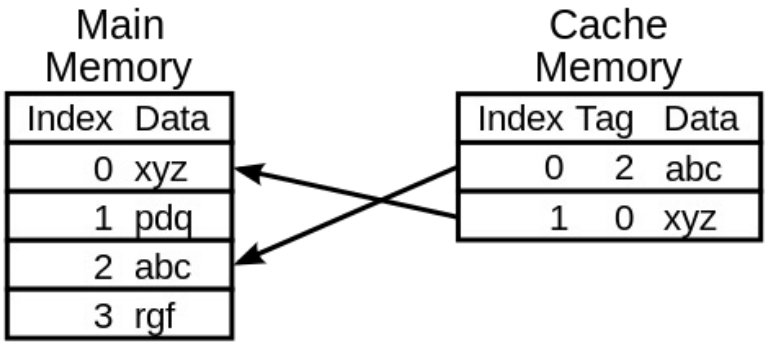
27

Cache



28

Cache



29

Replacing algorithm

It defines which cache line should be used to store a memory block, amongst those associated with the block (in the case of associative or set associative mapping).

It is chosen from:

- ***LRU*** **(Least Recently Used): the most used**
- ***FIFO*** **(First-In First-Out): the cheapest**
- ***LFU*** **(Least Frequently Used): theoretically, the most effective**
- ***random*****: simple and effective.**

30

Main memory update

When a write operation is performed on a data in the cache, we also need to update the main memory.

Two solutions can be adopted:

- ***write-back***
- ***write-through.***

31

Write-Back

For each cache block, a flag (called *dirty bit*) is introduced, which remembers whether or not the block has been changed since it was loaded into the cache.

When a block is deleted from the cache and the dirty bit is set, the block is copied from the cache to the main memory.

Disadvantages:

- **the replacement is slower because it sometimes requires copying in the main memory the replaced block**
- **in multiprocessor systems there may be incoherence between the caches of different processors**
- **it may not be possible to restore memory data after possible system failures.**

32

Write-through

Each time the CPU performs a write operation, it writes on both the cache data and the main memory data.

The resulting loss of efficiency is limited by the fact that writing operations are usually much less numerous than reading ones.

33

Block size

As block size increases (and with the same overall cache size), there are 2 phenomena:

- **first the hit ratio grows (thanks to the locality of references)**
- **then it begins to decrease (because the number of cache blocks decreases).**

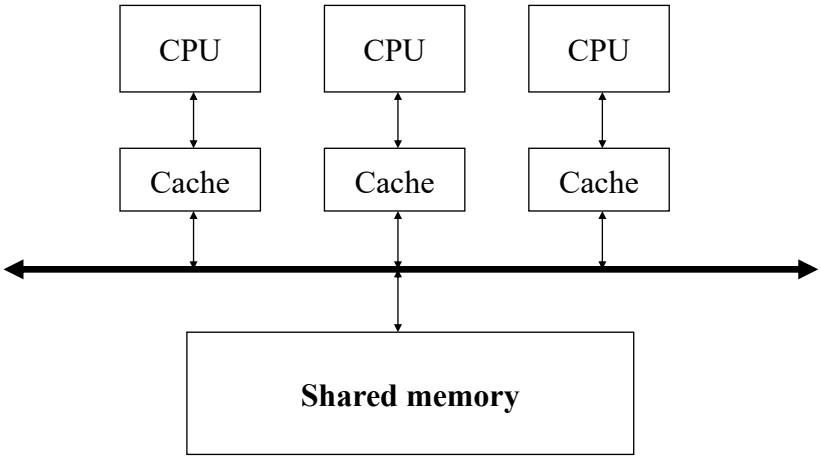
Frequent values are from 4 to 32 bytes.

34

Cache coherence

This is a major problem in multiprocessor systems with shared memory, in which each processor has its own cache.

Similar problems may occur if the system uses a DMA controller.



35

35

Validity bit

To achieve cache coherence, a *validity bit* is introduced for every cache line.

If it is disabled, it means that any access to the block must produce a miss.

At the power-on, the validity bits of all cache lines are disabled.

36

36

First, second, third level caches

It may be convenient to have multiple levels of caches:

- **a first level cache (L1), smaller and faster**
- **a second level cache (L2), slower but larger**
- **a third level cache (L3), even slower and bigger.**

Caveats

- **L1 is always SRAM**
- **Modern L2 & L3 are "usually" SRAM, but not quite optimized for speed as L1**
- **L2 & L3 may be shared among cores**

37

First, second, third level caches: behavior

Each time the processor performs a memory access

- **It checks first whether the word is in L1**
- **If so, it can access the word in L1**
- **If not, it checks whether the word is in L2**
  - **If so, it will access to the word in L2 and eventually update L1**
  - **If not, it checks whether the word is in L3**
    - **If yes, L3 is accessed and L2 is updated**
    - **If not, it accesses to the main memory and eventually updates L3.**

38

39

Examples

- IBM Power5
  - L1I: 64kB 2-way 128B block LRU
  - L1D: 32kB 4-way 128B block write-through LRU
  - L2: 1.875MB (3 banks) 10-way 128B block pseudo LRU
- Intel Core Duo
  - L1I: 32kB 8-way 64B block LRU
  - L1D: 32kB 8-way 64B block LRU write-through
  - L2: 2MB 8-way 64B line LRU write-back
- Sun Niagara
  - L1I: 16kB 4-way 32B block random
  - L1D: 8kB 4-way 16B block random write-through write no-allocate
  - L2: 3MB 4 banks 64B block 12-way write-back

40

Example of exercise

Let us consider a cache with the following characteristics:

- **64KByte size (data only)**
- **direct mapping**
- **4 bytes blocks**
- **32 bits addresses.**

You are asked to determine the structure of the cache (number of lines, size of the tag field).

41

Example of exercise

Since each block is compose of 4 bytes, the memory is composed of 2^{30} blocks.

The number of lines in the cache is 64KByte/4=16K=2^{14}.

The tag field identifies the block currently stored in each line.

Hence, the tag field should be composed of 30 bit.

However, since in the generic cache line only blocks whose index has the 14 least significant bits equal to the line index, the tag field is composed of 16 bits, only.

Hence, the total size of the cache is given by:

$$2^{14} \times (32 + 16) = 2^{14} \times 48 = 768\text{Kbit} = 96\text{KByte}$$

42